

Базы данных

Тема 14

СУБД SQL Server 2012 – Создание
распределенных баз данных.

Методы распределения данных по нескольким БД

- горизонтальная фрагментация:
 - таблица разбивается на несколько частей так, чтобы каждая часть содержала записи со значениями только из некоторого диапазона;
 - выполняется операцией селекции, распределяющей кортежи в соответствии с предикатом селекции;
- вертикальная фрагментация:
 - реализуется с применением операции проекции, необходимы дополнительные соединения для достижения значений нерезидентных атрибутов;
 - таблица разбивается на части так, чтобы каждая часть содержала только некоторые из столбцов исходной таблицы (разбиение строк);
- тиражирование:
 - на каждом сервере хранится копия исходной таблицы
 - тиражирование целесообразно, если менее 5% инструкций, ссылающихся на таблицу, - инструкции INSERT, UPDATE или DELETE.

Горизонтальная фрагментация

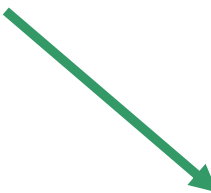
ID	Col A	Col B
0	...	...
...	...	...
999	...	...
1000	...	...
...	...	...
1999	...	...

ID < 1000



Сервер 1

ID	Col A	Col B
0	...	...
...	...	...
999	...	...



ID >= 1000

Сервер 2

ID	Col A	Col B
1000	...	...
...	...	...
1999	...	...

Создание горизонтальной фрагментации

- на каждом сервере:

```
CREATE TABLE Table (  
    ID INTEGER PRIMARY KEY CHECK (ID < 1000) ,  
    ColumnA INTEGER,  
    ColumnB INTEGER)
```

- на сервере, содержащем представление:

```
USE master
```

```
EXEC sp_serveroption 'OtherServer',  
    'lazy schema validation', 'true'
```

```
CREATE VIEW View AS  
    SELECT * FROM MyDB.dbo.Table  
    UNION ALL  
    SELECT * FROM OtherServer.MyDB.dbo.Table
```

Вертикальная фрагментация

ID	Col A	Col B
0	...	...
...	...	...
...	...	...
1999	...	...

ID, Col A

ID, Col B

Сервер 1	
ID	Col A
0	...
...	...
...	...
1999	...

Сервер 2	
ID	Col B
0	...
...	...
...	...
1999	...

Создание вертикальной фрагментации

- на первом сервере:

```
CREATE TABLE Table (  
    ID INTEGER PRIMARY KEY, ColumnA INTEGER)
```

- на втором сервере:

```
CREATE TABLE Table (  
    ID INTEGER PRIMARY KEY, ColumnB INTEGER)
```

- на сервере, содержащем представление:

```
CREATE VIEW View AS  
    SELECT one.ID, one.ColumnA, two.ColumnB  
    FROM MyDB.dbo.Table one,  
         OtherServer.MyDB.dbo.Table two  
    WHERE one.ID = two.ID
```

Секционирование в СУБД SQL Server 2012: федеративные серверы баз данных (federated database servers) / (distributed partitioned views)

- цель: обеспечение высокого уровня доступности путем создания федерации серверов баз данных (горизонтального секционирования данных базы), что позволит распределить нагрузку на группу серверов, управляющихся независимо, но взаимодействующих во время обработки запросов к базе данных;
- процесс построения федерации серверов баз данных включает разработку набора распределенных секционированных представлений, используемых для распределения данных по серверам;
- секционирование дает хорошие результаты, если строение таблиц в базе данных позволяет поделить их на одинаковые секции так, чтобы большинство строк, к которым обращается любая из инструкций SQL, были бы размещены на одном федеративном сервере.

Симметричные секции

- наиболее эффективно секционирование работает в случае, если все таблицы в базе данных могут быть секционированы симметрично (то есть равномерно по федеративным серверам), при этом:
 - взаимосвязанные данные размещаются на одном федеративном сервере таким образом, чтобы большинство инструкций SQL минимально нуждалось в данных, размещенных на других серверах;
 - цель проектирования распределенных секционированных представлений можно сформулировать как правило 80/20: большинство инструкций SQL должно обращаться к серверу, содержащему не менее 80% требуемых данных, а на распределенные запросы должно приходиться менее 20% данных;
 - в качестве проверки достижимости этой цели рекомендуется посмотреть, позволяет ли секция разместить на одном федеративном сервере все строки внешних ключей со всеми их ссылками;
- пример: секционирование данных по регионам
 - большинство запросов нуждается в данных с сервера одного региона;
 - приложения направляют инструкции SQL на федеративный сервер с данными того региона, который был определен из контекста входных данных.

Асимметричные секции

- большинство приложений имеют сложные схемы доступа к данным, не позволяющие осуществить симметричное секционирование;
- при асимметричном секционировании некоторым федеративным серверам приходится назначать более масштабные роли, чем другим - в базе данных могут быть секционированы лишь некоторые из таблиц, а остальные таблицы приходится оставлять на первоначальном сервере, при этом производительность для этих таблиц обычно повышается за счет перераспределения нагрузки на рассматриваемый сервер;
- асимметричное секционирование позволяет добиться большей части производительности, возможной во время симметричного секционирования, и при этом имеет следующие важные преимущества:
 - с помощью асимметричного секционирования некоторых таблиц можно значительно улучшить производительность баз данных, которые не могут быть секционированы симметрично;
 - большую существующую систему можно успешно секционировать посредством серии последовательных, асимметричных процедур секционирования;
- многие базы данных могут быть секционированы несколькими способами: для реализации должны выбираться те конкретные секции, которые наилучшим образом отвечают требованиям типичного набора инструкций SQL, выполняемых на уровне бизнес-служб.

Проектирование распределенных секционированных представлений

- при проектировании набора распределенных секционированных представлений для реализации федерации серверов баз данных, необходимо выполнить следующие процедуры:
 - определить шаблон инструкций SQL, выполняемых приложением;
 - определить, каким образом таблицы связаны друг с другом;
 - сопоставить частоту инструкций SQL с секциями, определенными при анализе внешних ключей;
 - определить правила маршрутизации инструкций SQL.

Шаблон инструкций SQL, выполняемых приложением

- создание шаблона инструкций SQL, выполняемых приложением, сводится к следующей последовательности шагов:
 - создание списка типичных инструкций SQL, которые будут выполняться приложением в течение большей части работы;
 - разбиение списка на категории SELECT, UPDATE, INSERT и DELETE;
 - сортировка инструкций в каждой из категорий по частоте выполнения, при этом, если инструкции SQL ссылаются на хранимые процедуры, необходимо включать в список базовые инструкции SELECT, INSERT, UPDATE и DELETE, содержащиеся в этих хранимых процедурах (если секционирована существующая база данных SQL Server, для получения этого списка можно воспользоваться SQL Server Profiler);
 - оценка соотношения числа вызовов и количества запрашиваемых данных для каждой инструкции, при этом каждая из инструкций SQL может быть отнесена к одному из двух типов:
 - инструкции, работающие с небольшими объемами данных (типичны для систем обработки транзакций);
 - инструкции, работающие с большими объемами данных (типичны для систем принятия решений (OLAP));

Связи между таблицами

- целью анализа связей между таблицами является нахождение кластеров таблиц, которые могут быть секционированы по одному измерению (например, по номеру детали, коду отдела или региону), чтобы все строки, относящиеся к конкретным значениям этого измерения, оказались на одном сервере;
- поскольку явные или неявные внешние ключи определяют связи между таблицами, они являются важнейшими элементами при поиске способа секционирования данных, так как показывают, как часто запросы будут пользоваться строками одной таблицы для поиска строк в другой;
- в качестве неявных рассматриваются ключи, не определенные непосредственно в составе схемы базы данных, например, неключевые поля таблиц, используемые для операций соединения в инструкциях SQL: такие неявные внешние ключи обычно индексируются, чтобы повысить производительность операций соединения.

Частота использования инструкций SQL применительно к секциям

- производится сопоставление частоты инструкций SQL с секциями, определенными при анализе внешних ключей и выбор секционирования, которое лучше всего поддерживает весь набор используемых в приложении инструкций SQL:
 - если некоторая часть таблиц может быть секционирована более чем одним способом, по частоте использования инструкций SQL необходимо определить, какие из секций удовлетворяют большему числу инструкций;
 - таблицы, которые чаще всего указываются в инструкциях SQL, нужно секционировать в первую очередь;
- шаблон инструкций SQL также влияет на принятие решения о секционировании / тиражировании таблиц:
 - таблица секционируется, если более 5% инструкций, ссылающихся на нее, - это инструкции INSERT, UPDATE или DELETE, и если таблица может быть секционирована по выбранному измерению;
 - сохраняются полные копии таблиц на каждом из серверов, если менее 5% инструкций, ссылающихся на таблицу, - инструкции INSERT, UPDATE или DELETE - при этом необходимо определить, каким образом производятся изменения, чтобы обновлялись все копии таблицы.
 - не секционируются и не копируются таблицы, если более 5% инструкций, ссылающихся на нее, являются инструкциями INSERT, UPDATE или DELETE, а таблица не может быть секционирована по выбранному измерению.

Правила маршрутизации инструкций SQL

- правила маршрутизации инструкций SQL должны обеспечивать определение сервера, который способен наиболее эффективно обработать каждую из инструкций SQL;
- правила маршрутизации должны установить связь между контекстом пользовательского ввода и сервером, который содержит данные, необходимые для выполнения инструкции;
- приложения должны быть способны принять данные, введенные пользователем, и сопоставить их с правилами маршрутизации, чтобы определить, какой из серверов должен обработать данную инструкцию SQL.

Горизонтальное секционирование

- прежде чем реализовывать секционированные представления, необходимо произвести горизонтальное секционирование: исходная таблица заменяется несколькими меньшими таблицами-элементами:
 - каждая таблица-элемент и исходная таблица имеют одинаковое количество столбцов, у каждого столбца такие же атрибуты, как и у соответствующего ему столбца в исходной таблице (тип данных, размер, параметры сортировки);
 - при создании распределенного секционированного представления каждая таблица-элемент находится на отдельном сервере;
- чтобы расположения были как можно прозрачнее, имена баз данных-элементов должны быть одинаковыми на каждом сервере-элементе, хотя это и не обязательно (например, **Server1.CustomerDB, Server2.CustomerDB, Server3.CustomerDB, ...**)

Создание таблиц-элементов

- таблицы-элементы конструируются так, чтобы в каждой таблице хранился горизонтальный фрагмент исходной таблицы, основанный на интервале значений ключа
- интервалы секционирования основаны на значениях данных в столбце секционирования: диапазон значений в каждой таблице-элементе определяется проверочным ограничением на столбце секционирования, и интервалы не пересекаются.

```
-- On Server1:
```

```
CREATE TABLE Customers_33  
  (CustomerID INTEGER PRIMARY KEY  
    CHECK (CustomerID BETWEEN 1 AND 32999),  
  ... -- Additional column definitions)
```

```
-- On Server2:
```

```
CREATE TABLE Customers_66  
  (CustomerID INTEGER PRIMARY KEY  
    CHECK (CustomerID BETWEEN 33000 AND 65999),  
  ... -- Additional column definitions)
```

```
-- On Server3:
```

```
CREATE TABLE Customers_99  
  (CustomerID INTEGER PRIMARY KEY  
    CHECK (CustomerID BETWEEN 66000 AND 99999),  
  ... -- Additional column definitions)
```


Определение распределенных секционированных представлений

- после создания таблиц-элементов на каждом сервере-элементе при построении распределенных секционированных представлений выполняются следующие задачи:
 - добавление определений связанных серверов на каждый сервер-элемент, содержащий сведения о соединении, необходимые для запуска распределенных запросов на других серверах-элементах;
 - установка параметра **пассивная проверка схемы** для всех определений связанных серверов, применяемых в распределенных секционированных представлениях (с помощью процедуры **sp_serveroption**), что позволяет оптимизировать производительность;
 - создание распределенного секционированного представления на всех серверах-элементах:
 - представления обращаются к данным из связанных серверов-элементов с помощью распределенных инструкций SELECT и соединяют распределенные строки со строками из локальной таблицы-элемента;
 - у всех представлений задается одно и то же имя: это позволяет запросам, которые ссылаются на имя распределенного секционированного представления, запускать один или несколько серверов-элементов (расположение данных прозрачно для приложения)

Основные требования к представлениям для распределенного горизонтального секционирования

- запросы должны быть соединены с помощью UNION ALL;
- базовые таблицы не должны входить в запрос более одного раза;
- столбцы должны быть одного типа, идти в одном и том же порядке и встречаться только один раз;
- столбец, определяющий сегментирование:
 - не должен быть вычисляемым;
 - должен входить в первичный ключ;
 - должен иметь единственное ограничение CHECK с операторами [BETWEEN, AND, OR, <, <=, >, >=, =] , определяющее набор непересекающихся диапазонов значений.

Создание распределенных секционированных представлений: пример

```
-- On Server1:
CREATE TABLE Customers_33
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 1 AND 32999) ,
  ... -- Additional column definitions)

-- On Server2:
CREATE TABLE Customers_66
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 33000 AND 65999) ,
  ... -- Additional column definitions)

-- On Server3:
CREATE TABLE Customers_99
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 66000 AND 99999) ,
  ... -- Additional column definitions)

-- On each server:
CREATE VIEW Customers AS
  SELECT * FROM CompanyDatabase.TableOwner.Customers_33
  UNION ALL
  SELECT * FROM Server2.CompanyDatabase.TableOwner.Customers_66
  UNION ALL
  SELECT * FROM Server3.CompanyDatabase.TableOwner.Customers_99
```

Правила таблиц-элементов секционированных представлений

- на таблицы-элементы нельзя ссылаться в представлении больше одного раза
- у таблиц-элементов не может быть индексов, созданных на любых вычисляемых столбцах;
- все ограничения первичного ключа таблиц-элементов должны быть на одинаковом количестве столбцов;
- у таблиц-элементов должна быть одна и та же настройка дополнения ANSI (параметр конфигурации ANSI_PADDING).

Правила столбцов таблиц-элементов секционированных представлений

- все столбцы в каждой таблице-элементе должны быть включены в список выборки (допустимый синтаксис: `SELECT * FROM <таблица-элемент>`);
- на столбцы нельзя ссылаться в списке выборки больше одного раза;
- в списке выборки у столбцов должен быть такой же порядок следования;
- тип столбцов в списке выборки каждой инструкции `SELECT` должен совпадать: это относится к типам данных, точности, масштабу и параметрам сортировки

Правила столбцов секционирования

- для секционирования может использоваться только один столбец, и он должен присутствовать в каждой таблице-элементе;
- проверочные ограничения определяют данные, доступные в каждой таблице-элементе, при этом:
 - диапазон ключа проверочных ограничений в таблице не должен пересекаться с диапазонами любой другой таблицы;
 - проверочные ограничения могут использовать только следующие операторы: BETWEEN, IN, AND, OR, <, <=, >, >=, =;
- столбец секционирования должен быть частью первичного ключа таблицы;
- столбец секционирования должен находиться в одинаковом порядковом расположении в списках выборки всех инструкций SELECT в представлении;
- столбец секционирования не может быть столбцом с автоматическим приращением, столбцом по умолчанию или столбцом типа **timestamp**;
- столбец секционирования не может содержать значения NULL;
- столбец секционирования не может быть вычисляемым столбцом;
- у столбца секционирования может быть только одно ограничение (если задано несколько ограничений, то SQL Server игнорирует их все и не учитывает их при выяснении того, является ли это представление секционированным);
- ограничения для обновления столбца секционирования отсутствуют.

Изменение данных в секционированных представлениях

- обновляемое секционированное представление обладает всеми возможностями исходной таблицы;
- представление считается обновляемым секционированным, если:
 - оно представляет собой набор инструкций SELECT, результирующие наборы которых объединяются в один набор инструкцией UNION ALL;
 - каждая инструкция SELECT ссылается на одну базовую таблицу SQL Server (локальную или связанную);
 - в базовых таблицах отсутствуют триггеры и каскадные действия, т.к. они могут внести изменения в базовые данные, влияющие на определение представления;
- инструкции, изменяющие данные и ссылающиеся на представление, должны следовать правилам, определенным для инструкций INSERT, UPDATE и DELETE.

Инструкции INSERT

- инструкции INSERT добавляют данные в базовые таблицы через секционированное представление и должны следовать следующим правилам:
 - в инструкцию INSERT должны включаться все столбцы, даже если в базовой таблице значением столбца может быть NULL или если для него определено ограничение DEFAULT;
 - ключевое слово DEFAULT в предложении VALUES инструкции INSERT использовать нельзя;
 - в инструкциях INSERT должны содержаться значения, удовлетворяющие логике ограничения CHECK, определенного для секционированного столбца одной или более базовых таблиц;
 - инструкции INSERT запрещены, если:
 - в одной из базовых таблиц есть столбец типа **timestamp**;
 - есть самосоединение с тем же самым представлением или с одной из базовых таблиц.

Инструкции UPDATE и DELETE

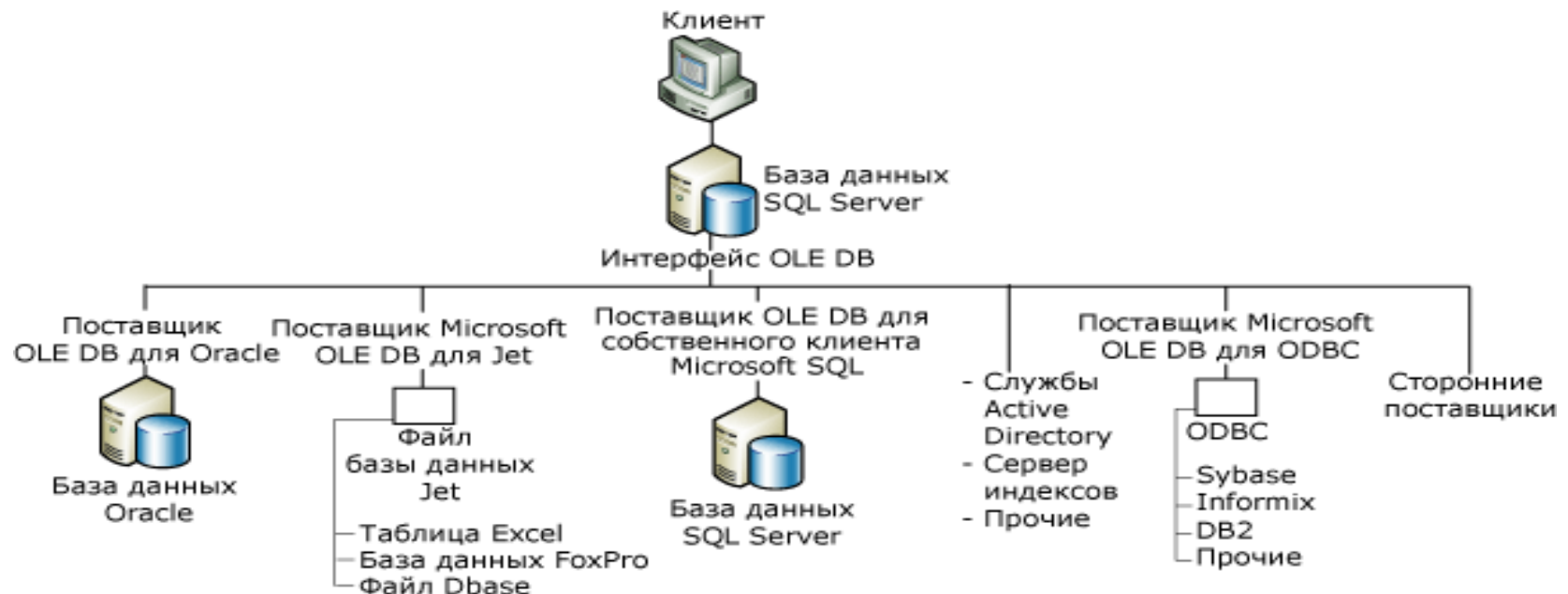
- инструкции UPDATE изменяют данные в одной или более базовых таблицах через секционированное представление, при этом они должны следовать следующим правилам:
 - ключевое слово DEFAULT в предложении SET инструкции UPDATE использовать нельзя, даже если в соответствующей базовой таблице для данного столбца определено значение DEFAULT;
 - значение столбца с ограничением PRIMARY KEY может быть изменено, если он содержит данные типа **text**, **image** или **ntext**;
 - инструкции UPDATE запрещены, если:
 - в одной из базовых таблиц есть столбец типа **timestamp**;
 - есть самосоединение с тем же самым представлением или с одной из базовых таблиц;
- инструкции DELETE удаляют данные в одной или более базовых таблицах через секционированное представление, и запрещены, если есть самосоединение с тем же самым представлением или с одной из базовых таблиц;

Тиражирование

- тиражирование целесообразно, если менее 5% инструкций, ссылающихся на таблицу, - инструкции INSERT, UPDATE или DELETE;
- в зависимости от требований к целостности можно использовать два подхода к обновлению данных:
 - если требуется высокая целостность транзакций, можно создать триггеры, выполняющие распределенное обновление всех копий в контексте распределенной транзакции;
 - если высокая целостность не требуется, можно для переноса изменений от одной копии таблицы на все остальные воспользоваться механизмами репликации SQL Server;
- может обеспечиваться с помощью механизмов репликации:
 - репликация *моментальных снимков* (snapshot replication) распространяет данные подписчикам целиком и точно в том виде, в котором они были представлены в момент синхронизации, и не контролирует обновления этих данных;
 - *транзакционная репликация* (transactional replication) начинается с создания исходного моментального снимка объектов и данных базы данных публикации, последующие изменения данных и схемы на издателе обычно доставляются подписчику без задержек (практически в реальном времени), а изменения данных применяются на подписчике в том же порядке и в тех же рамках транзакций, в которых они выполнялись у издателя (в пределах публикации гарантируется согласованность транзакций).

Распределенные запросы

- распределенные запросы используются для доступа к данным из нескольких гетерогенных источников данных, которые могут размещаться на одном или различных компьютерах;
- Microsoft SQL Server 2012 поддерживает распределенные запросы к следующим типам источников данных:
 - распределенные данные, хранящиеся в нескольких экземплярах SQL Server;
 - гетерогенные данные, хранящиеся в различных реляционных и нереляционных источниках данных, доступ к которым осуществляется с использованием поставщика OLE DB (которые представляют данные в табличных объектах, именуемых «наборами строк»)



Стандартные OLE DB провайдеры

- SQL Server (Microsoft)
- ODBC (Microsoft)
- Microsoft Jet (Microsoft)
- Oracle (Microsoft)
- Oracle (Oracle)
- DB2 (Microsoft)
- Microsoft Directory Services
- Microsoft Indexing Service
- Exchange Server
(Microsoft)

Связанный сервер (Linked server)

- связанным называется сервер, зарегистрированный на SQL Server 2012 со всеми сведениями, необходимыми для доступа к источнику данных OLE DB, включая данные о:
 - поставщике OLE DB - динамической библиотеке, осуществляющей управление и взаимодействие с определенными источниками данных;
 - источнике данных OLE DB - конкретной базе данных, доступ к которой выполняется через интерфейс OLE DB;
- доступ к связанному серверу может осуществляться следующими способами:
 - через распределенные запросы, обращающиеся к таблицам на связанном сервере при помощи инструкций SELECT, INSERT, UPDATE и DELETE с именем, относящимся к такому серверу;
 - через удаленные хранимые процедуры, выполняющие обращения к связанному серверу по имени, состоящему из четырех частей;
 - через инструкцию EXECUTE, реализуемую произвольной параметризованной передаваемой командой при помощи расширения AT *linked_server_name*.

Ссылка на объекты связанного сервера

- для ссылки на объекты связанного сервера после его определения, в инструкциях Transact-SQL можно использовать полное имя из четырех частей вида *linked_server_name.catalog.schema.object_name*,
 - *linked_server_name* : связанный сервер, ссылающийся на источник данных OLE DB.
 - *catalog* : каталог источника данных OLE DB, содержащий объект.
 - *schema* : схема каталога, содержащего объект.
 - *object_name* : объект данных в схеме.
- параметры *catalog*, *schema* и *object_name* представляют собой ссылку на объект в связанном сервере, представляемый как набор строк, и передаются поставщику OLE DB для идентификации конкретного объекта данных.
- когда связанный сервер ссылается на экземпляр SQL Server, параметр *catalog* ссылается на базу данных, а параметр *schema* - на схему.
- на удаленные таблицы можно ссылаться по имени связанного сервера одним из следующих способов:
 - по имени сервера в состоящей из четырех частей ссылке на имя таблицы или представления в инструкции языка Transact-SQL;
 - с помощью входного параметра функции OPENQUERY. Функция OPENQUERY передает поставщику OLE DB команду на выполнение. После этого возвращенный набор строк может быть использован как ссылка на таблицу или представление в инструкции языка Transact-SQL.

Операции со связанными серверами

- получение списка зарегистрированных серверов:

sp_linkedservers

sys.servers (представление системного каталога)

- добавление / удаление сервера:

sp_addlinkedserver

sp_dropserver

sp_addlinkedserver

@server = 'LondonPayroll2',

@srvproduct = ' ',

@provider = 'SQLNCLI',

@provstr = 'Server=London\Payroll'

- добавление/удаление входного имени
(сопоставление имени и пароля удаленного входа
для заданного локального входа):

sp_addlinkedsrvlogin

sp_droplinkedsrvlogin

Создание связанных серверов: пример

```
EXEC sp_addlinkedserver 'SEATTLESales', N'SQL Server'
```

```
EXEC sp_addlinkedserver  
    @server='S1_instance1',  
    @srvproduct='',  
    @provider='SQLNCLI',  
    @datasrc='S1\instance1\  
    @catalog = 'Northwind'
```

```
EXEC sp_addlinkedserver  
    @server = 'LONDON Mktg',  
    @srvproduct = 'Oracle',  
    @provider = 'MSDAORA',  
    @datasrc = 'MyServer'
```

```
EXEC sp_droplinkedsrvlogin  
    @rmtsrvname = 'ServerOne',  
    @locallogin = NULL
```

```
EXEC sp_addlinkedsrvlogin  
    @rmtsrvname = 'ServerOne',  
    @useself = 'false',  
    @locallogin = NULL,  
    @rmtuser = 'UserName',  
    @rmtpassword = 'UserPassword'
```


Нерегламентированные имена (специальные серверы, ad hoc servers)

- в SQL Server макрокоманды OPENROWSET и OPENDATASOURCE предоставляют сведения для подключения, выполняемого с целью доступа к данным из источников данных OLE DB:
 - OPENDATASOURCE и OPENROWSET не предоставляют всех возможностей определений связанных серверов (например, управление безопасностью, управление сопоставлением имен входа, возможность запрашивать данные из каталогов, возможность настройки различных параметров соединения);
 - каждый раз при вызове этих функций должна предоставляться вся информация для подключения, в том числе пароли;
 - OPENROWSET и OPENDATASOURCE должны использоваться только для ссылки на источники данных OLE DB, обращения к которым происходят нечасто - для всех источников данных, к которым будет производиться достаточно большое количество обращений, необходимо определить связанный сервер;
 - OPENROWSET и OPENDATASOURCE не поддерживают предоставление переменных языка Transact-SQL в качестве аргументов (если в качестве аргумента требуется передать переменную, то строку запроса, содержащую переменные, можно сформировать динамически и выполнить при помощи инструкции EXECUTE).

OPENROWSET и OPENDATASOURCE

- функция OPENROWSET ...
 - может использоваться с любым поставщиком OLE DB, возвращающим набор строк;
 - может применяться везде, где в инструкции языка Transact-SQL используется ссылка на таблицу или представление;
 - требует задания следующих данных:
 - всех сведений, необходимых для подключения к источнику данных OLE DB;
 - имени объекта или запроса, которые должны формировать набор строк.
- функция OPENDATASOURCE ...
 - применяется только в тех случаях, если поставщик предоставляет наборы строк и использует нотацию *каталог.схема.объект*;
 - может использоваться там, где синтаксис Transact-SQL позволяет указать имя связанного сервера;
 - может использоваться в нотации *каталог.схема.объект* в качестве ссылки на таблицу или представление;
 - требует задания следующих данных:
 - зарегистрированного как PROgid имени поставщика OLE DB, используемого для доступа к источнику данных;
 - строки соединения, задающей различные свойства соединения, передаваемые поставщику OLE DB (последовательность пар "ключевое слово-значение").

Специальные серверы: синтаксис

```
OPENDATASOURCE (  
    'provider_name',  
    'init_string'  
)
```

```
OPENROWSET (  
    'provider_name',  
    { 'datasource' ; 'user_id' ; 'password'  
      | 'provider_string' },  
    { [catalog.] [schema.] object  
      | 'query' }  
)
```

Использование специальных серверов

```
SELECT * FROM
    OPENDATASOURCE('SQLNCLI',
        'Data Source=MyServer\Instance1;
        User ID=UserName;Password=UserPassword'
    ).Northwind.dbo.Customers
```

```
SELECT * FROM
    OPENROWSET('SQLNCLI',
        'MyServer\Instance1'; 'UserName'; 'UserPassword',
        Northwind.dbo.Customers)
```

```
SELECT a.* FROM
    OPENROWSET
    ('SQLNCLI', 'Server=Seattle1;Trusted_Connection=yes;',
        'SELECT GroupName, Name, DepartmentID
        FROM AdventureWorks.HumanResources.Department
        ORDER BY GroupName, Name'
    )
    AS a;
```

Сквозные запросы

- функции OPENQUERY позволяют выполнить передаваемый запрос к указанному связанному серверу:
 - на функцию OPENQUERY как на имя таблицы можно ссылаться из предложения FROM, в инструкциях INSERT, UPDATE или DELETE;
 - в качестве аргументов нельзя использовать переменные;
 - функция OPENQUERY не может быть использована для выполнения расширенных хранимых процедур на связанном сервере.

```
OPENQUERY (  
    'linked_server',  
    'query'  
)
```

```
EXEC sp_addlinkedserver 'OracleSvr', 'Oracle 7.3', 'MSDAORA', 'ORCLDB'  
GO
```

```
SELECT * FROM  
    OPENQUERY('OracleSvr', 'SELECT name, id FROM joe.titles')  
GO
```

Уровни диалекта SQL

(уровни поддержки SQL провайдерами)

- существуют следующие уровни диалекта, в порядке убывания от старшего к младшему (каждый из перечисленных уровней диалекта является надмножеством более низких уровней):
 - SQL Server
 - внешние соединения (OUTER JOIN), CUBE, ROLLUP, оператор получения остатка от деления (%), битовые операторы (&, |, ^, ~), строковые и системные арифметические функции;
 - SQL-92 начального уровня (SQL-92 Entry Level)
 - UNION, UNION ALL;
 - базовый ODBC(ODBC Core)
 - агрегационные функции с ключевым словом DISTINCT и символьные константы;
 - Jet (SQL Minimum)
 - Базовые конструкции SQL: статистические функции без ключевого слова DISTINCT, сортировка (ORDER BY), внутренние соединения (INNER JOIN), предикаты, операторы вложенных запросов (EXISTS, ALL, SOME, IN), DISTINCT, арифметические операции и константы, не упомянутые на более высоких уровнях, и логические операторы;
- дополнительные свойства SQL, поддерживаемые провайдерами:
 - поддержка GROUP BY и HAVING;
 - поддержка подзапросов (вложенных запросов);
 - поддержка констант даты/времени;
 - поддержка LIKE;
 - поддержка внутренних соединений;
 - поддержка параметрических запросов (маркеров параметров).

Оптимизация распределенных запросов

- для повышения производительности SQL Server 2012 выполняет следующие оптимизации, относящиеся к распределенным запросам:
 - индексированный доступ при помощи поставщиков индексов OLE DB: SQL Server может пользоваться стратегиями выполнения, которые включают применение индексов от поставщика индексов для вычисления предикатов и выполнения операций сортировки для удаленных таблиц;
 - удаленное выполнение запроса при помощи поставщиков OLE DB SQL-команд: SQL Server пытается делегировать выполнение как можно большей части распределенного запроса поставщику SQL-команд.

Удаленное выполнение распределенных запросов

- запрос, который производит доступ только к удаленным таблицам, находящимся в источнике данных поставщика, извлекается из исходного распределенного запроса и выполняется поставщиком (для минимизации числа строк, возвращаемых от поставщика, и обеспечения возможности при выполнении запроса пользоваться индексами).
- на степень делегирования выполнения распределенного запроса влияют следующие факторы:
 - уровень диалекта, поддерживаемый поставщиком SQL-команд: SQL Server делегирует только те операции, которые поддерживаются имеющимся уровнем диалекта;
 - совместимость параметров сортировки: для распределенного запроса семантика сравнения для всех символьных данных определяется кодировкой и порядком сортировки локального экземпляра SQL Server. SQL Server интерпретирует свойство режима сопоставления символьных данных из удаленного источника данных и обрабатывает его соответствующим образом. SQL Server может делегировать операции сравнения и ORDER BY для символьных столбцов поставщику только в том случае, если тот сможет определить, что
 - столбец в источнике данных имеет ту же кодировку и параметры сортировки;
 - семантика сравнения символов соответствует стандартам SQL-92 и SQL Server.

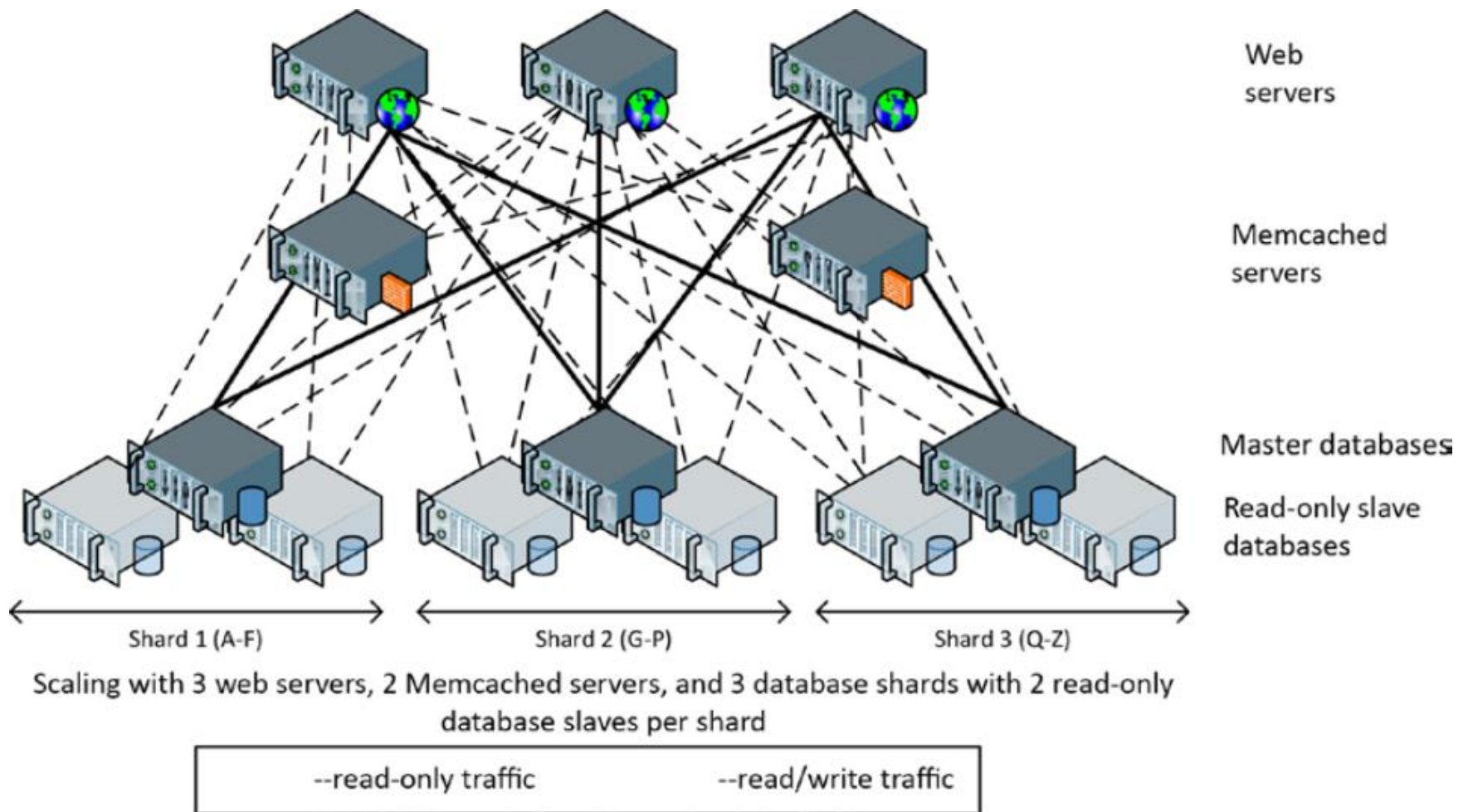
Распределенные транзакции

- участники распределенной транзакции:
 - два или более сервера, которые называются *диспетчерами ресурсов* (экземпляры SQL Server Database Engine);
 - *диспетчер транзакций*, который обеспечивает управление транзакцией путем координации между диспетчерами ресурсов (координатор распределенных транзакций (MS DTC) или другой диспетчер транзакций, поддерживающий спецификацию X/Open XA обработки распределенных транзакций);
- двухфазовая фиксация транзакций (two-phase commit protocol - 2PC):
 - *фаза подготовки*: каждый из диспетчеров ресурсов обеспечивает устойчивость транзакции, в частности, записывая журнал транзакции на диск;
 - *фаза фиксации*: начинается в случае успешного завершения подготовки всеми диспетчерами ресурсов и завершается после получения диспетчером транзакций подтверждений об успешной фиксации от каждого из диспетчеров ресурсов;
- управление распределенными транзакциями может осуществляться через вызов инструкций Transact-SQL (как непосредственно, так и через ADO, OLE DB и ODBC), а также через вызов функций API OLE DB и ODBC.

Процедура выполнения распределенной транзакции в Transact-SQL

1. приложение или сценарий Transact-SQL выполняет инструкцию Transact-SQL, которая инициирует распределенную транзакцию:
 - явный запуск распределенной транзакции: **BEGIN DISTRIBUTED TRANSACTION**;
 - выполнение распределенного запроса внутри локальной транзакции – транзакция становится распределенной (таким образом, если источник данных не поддерживает интерфейс **ITransactionJoin**, допустимы лишь инструкции только для чтения);
 - выполнение удаленной хранимой процедуры внутри локальной транзакции (**SET REMOTE_PROC_TRANSACTIONS ON**);
2. экземпляр SQL Server Database Engine, выполняющий инструкцию, становится управляющим сервером в транзакции;
3. сценарий или приложение выполняет либо распределенные запросы к связанным серверам, либо удаленные хранимые процедуры на удаленных серверах;
4. после того как распределенные запросы и вызовы удаленных процедур выполнены, управляющий сервер автоматически вызывает координатор распределенных транзакций Microsoft (MS DTC), чтобы прикрепить в распределенную транзакцию связанные и удаленные серверы;
5. когда сценарий или приложение выполняет инструкцию COMMIT или ROLLBACK, управляющий экземпляр SQL Server вызывает MS DTC, который управляет процессом двухфазной фиксации или уведомляет связанные и удаленные серверы, чтобы они выполнили откат своих транзакций.

Сегментирование данных в web- системах



Задание 13. Создание распределенных баз данных на основе секционированных представлений

1. Создать две базы данных на одном экземпляре СУБД SQL Server 2012.
2. Создать в базах данных п.1. горизонтально фрагментированные таблицы.
3. Создать секционированные представления, обеспечивающие работу с данными таблиц (выборку, вставку, изменение, удаление).

Задание 14. Создание вертикально фрагментированных таблиц средствами СУБД SQL Server 2012

1. Создать в базах данных пункта 1 задания 13 таблицы, содержащие вертикально фрагментированные данные.
2. Создать необходимые элементы базы данных (представления, триггеры), обеспечивающие работу с данными вертикально фрагментированных таблиц (выборку, вставку, изменение, удаление).

Задание 15. Создание распределенных баз данных со связанными таблицами средствами СУБД SQL Server 2012

1. Создать в базах данных пункта 1 задания 13 связанные таблицы.
2. Создать необходимые элементы базы данных (представления, триггеры), обеспечивающие работу с данными связанных таблиц (выборку, вставку, изменение, удаление).

Вопросы к экзамену

- Методы распределения данных: горизонтальная фрагментация.
- Методы распределения данных: вертикальная фрагментация.
- Методы распределения данных: тиражирование. Связанные таблицы. Обновление данных.
- Федеративные серверы баз данных. Симметричное и асимметричное секционирование.
- Проектирование распределенных секционированных представлений.
- Создание распределенных секционированных представлений в СУБД SQL Server 2012.
- Связанные серверы. Нерегламентированные имена (OPENDATASOURCE, OPENROWSET). Сквозные запросы (OPENQUERY).
- Распределенные запросы. Доступ к удаленным данным: поставщики OLE DB. Ограничения распределенных запросов. Оптимизация и выполнение распределенных запросов. Уровни диалекта SQL.
- Транзакции в SQL Server: распределенные транзакции.